

SML Assignment 4

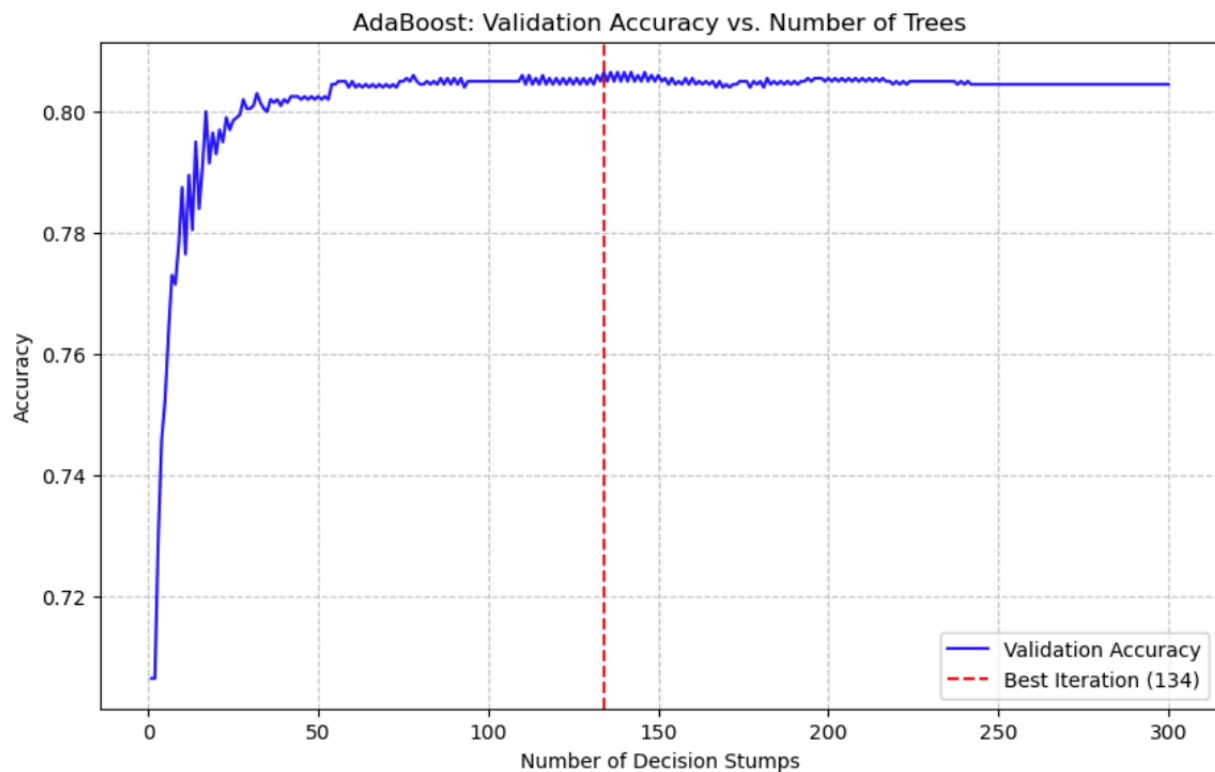
Mayank Yadav | 2024343

Question 1

Code Explanation

1. Loaded data for classes 4 and 9, assigned label -1 to class 4 and +1 to class 9.
2. Selected 1000 samples for validation and remaining for the train set.
3. Applied PCA to reduce dimension to 5, computing w for PCA using the train set.
4. Assigned equal weight to all train points for training.
5. Each decision stump (single split) is computed by minimizing the weighted error of the predictions by looping over 1000 randomly selected midpoints for each of the 5 features (candidates for the split threshold).
6. α ($\frac{1}{2} \ln(1-\text{err}/\text{err})$) is computed for a stump using the weighted error rate, and weights are updated for the next round for the misclassified samples by scaling them with $\exp(2 \alpha)$. (weights kept same for correctly classified samples).
7. Validation set accuracy is measured for the ensemble model of all stumps after computing each new stump.
8. Above is repeated for 300 iterations.
9. The model was tested on the test set for the number of trees in the ensemble model which gave us the highest validation accuracy.

Plot & Metrics



Validation Set Accuracy on Ensemble with 134 trees: 80.65%

Test Set Accuracy on Ensemble with 134 trees: 79.71%

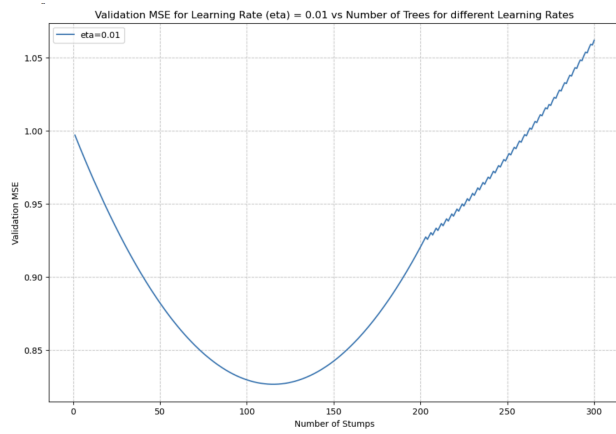
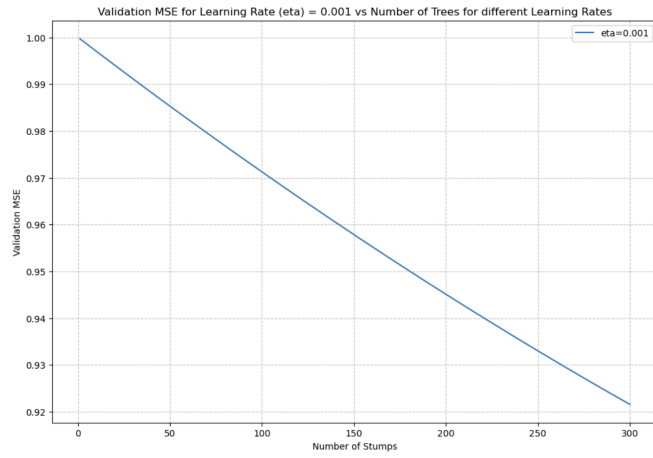
The accuracy improves drastically for increasing the number of trees as the model reduces bias and fits the data better, but after a point, the accuracy is saturated as the model begins to overfit and the newly added trees do not improve the accuracy.

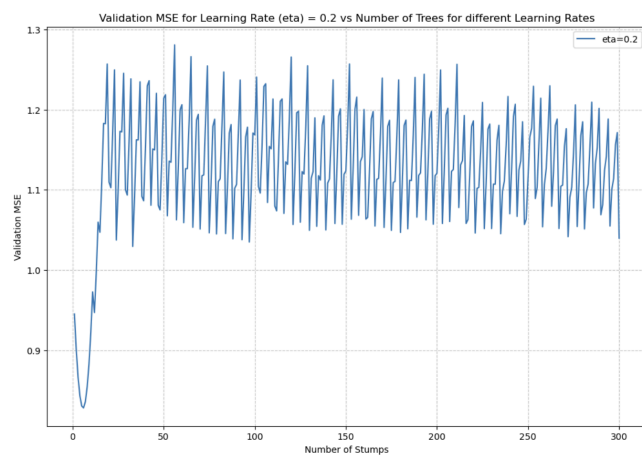
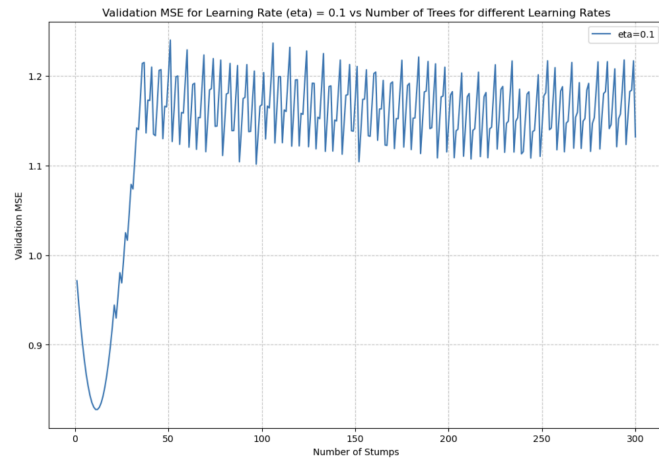
Question 2

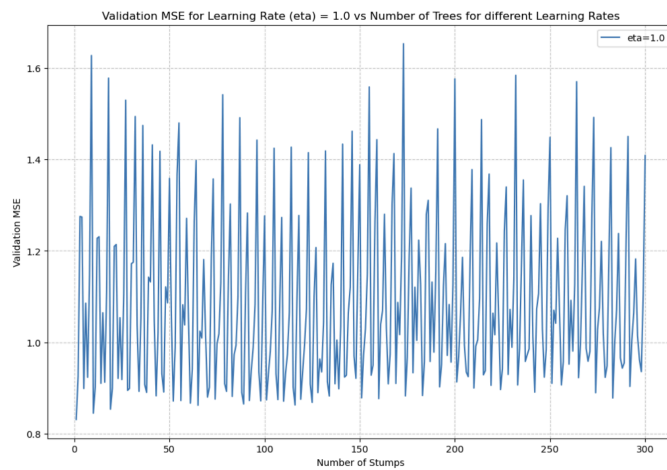
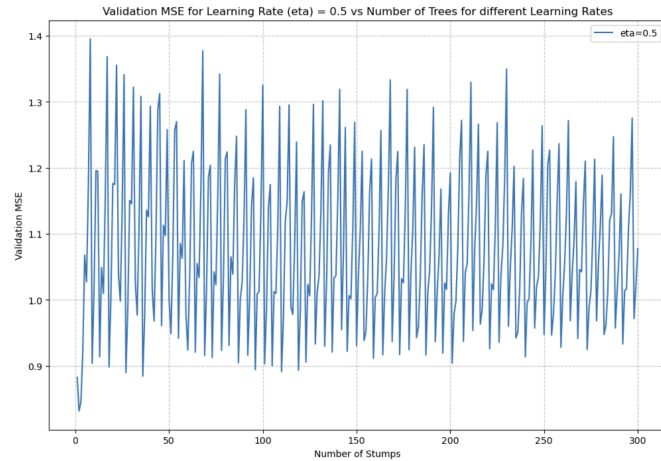
Code Explanation

1. Loaded data for classes 4 and 9, assigned label -1 to class 4 and +1 to class 9.
2. Selected 1000 samples for validation and remaining for the train set.
3. Applied PCA to reduce dimension to 5, computing w for PCA using the train set.
4. Assigned equal weight to all train points for training.
5. Each decision stump (single split) is computed by minimizing the ssr of the predictions with respect to the pseudo residuals computed for absolute loss (sign of label - prediction) by looping over 1000 randomly selected midpoints for each of the 5 features (candidates for the split threshold).
6. After each iteration, the labels are updated for the train set according to the negative gradient of the absolute loss.
7. The prediction for the ensemble is updated by learning rate * prediction of the current tree (so the residue becomes $y - F(x)$ where $F(x) = \eta_1 h_1(x) + \eta_2 h_2(x) + \dots + \eta_k h_k(x)$ for ensemble of k trees).
8. MSE is computed for the validation set and plotted against the number of trees.
9. Test error is com
10. The same is repeated for different values of η .

Plot & Metrics







Eta: 0.001	Best Tree: 300	Min Val MSE: 0.9215475545570905	Test MSE: 0.9289449199334088
Eta: 0.01	Best Tree: 116	Min Val MSE: 0.826840309592183	Test MSE: 0.8573252865506831
Eta: 0.1	Best Tree: 11	Min Val MSE: 0.8276293934679482	Test MSE: 0.8556392442867358
Eta: 0.2	Best Tree: 6	Min Val MSE: 0.8283309034795748	Test MSE: 0.8585166455566134
Eta: 0.5	Best Tree: 2	Min Val MSE: 0.8316956437416081	Test MSE: 0.8555061003437245
Eta: 1.0	Best Tree: 1	Min Val MSE: 0.8310488380352389	Test MSE: 0.8555135722214137

For low value of learning rate (0.001), the step size for gradient descent is too small, though the MSE goes down for every iteration, it will be a long time and lot more than 300 iterations before the optimal value is reached, so the model has the worst train and test MSE as it has not yet reached the minima (it still underfits the data)

For values of 0.01, 0.1, 0.2, the step size/learning rate increases, and the model first improves (underfitting to optimal number of trees), and then worsens (optimal value to overfitting), and after a point, the MSE value increases and decreases rapidly in short intervals, so the newly trained trees do not improve the model. Lowest validation MSE is observed for learning rate = 0.01, while the lowest test MSE is observed for learning rate = 0.5.

For values of 0.5, 1, the step size is too large, and because of this it may skip over the minimum (lowest validation MSE) and keep oscillating. After reaching the optimal number of trees very fast (2 and 1 tree respectively for 0.5 and 1) the newly trained trees do not improve the model because the first few steps already took aggressive steps.

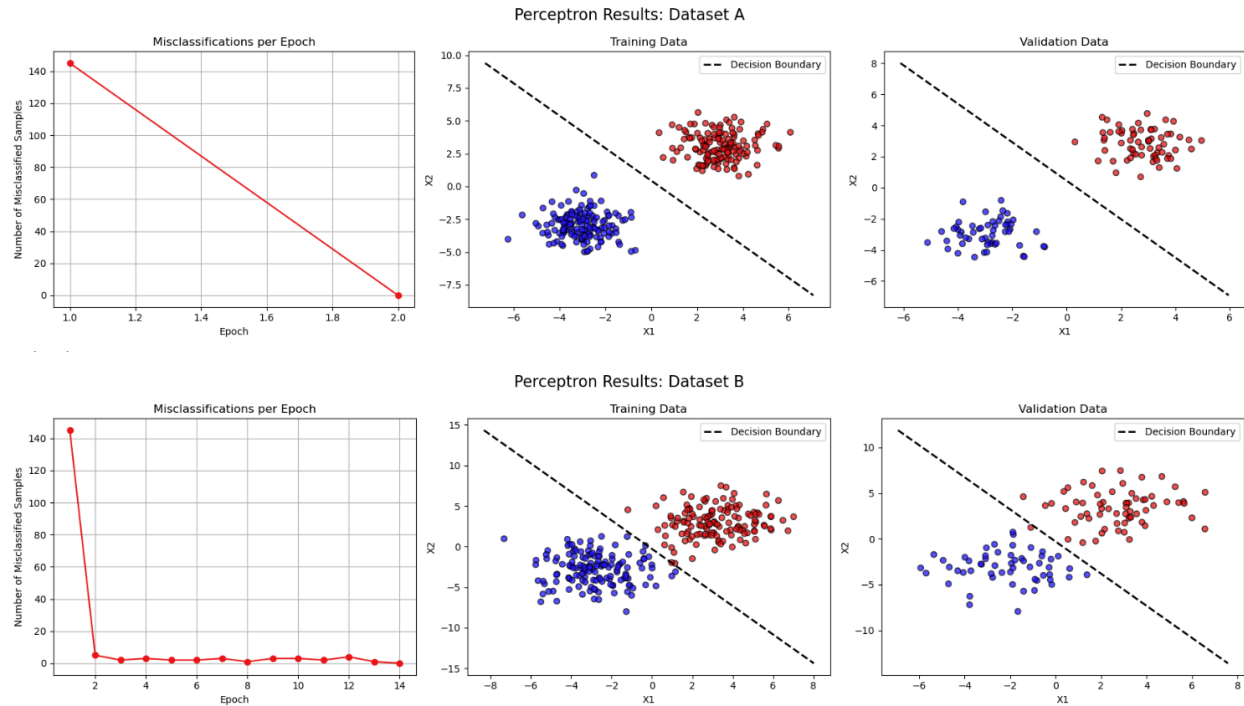
However, for learning rates 0.01, 0.1, 0.2, 0.5 and 1, the validation and test set MSEs are very close together, being within +/- 0.5 of each other.

Question 3

Code Explanation

1. Used `np.random.multivariate_normal()` to generate samples for dataset A and B, and split into training and test sets using a 70:30 train:test split.
2. The class with negative mean was given label -1, and with positive mean was given label +1.
3. Initialised weights and bias for the perceptron to be zero.
4. Perceptron predicts sign ($\text{weight} \cdot \text{point} + \text{bias}$) as the prediction, with sign zero being assigned to class 1, and signs -1, 1 being assigned to respective classes.
5. For a given iteration, predict the labels for all train points using the current weight and bias, and store all the misclassified points.
6. For updating the weights and bias, we follow stochastic gradient descent, randomly select any one misclassified point, and update the perceptron parameters using the negative gradient of the perceptron loss function at that point times the learning rate (0.01).
7. Perceptron loss is zero for correctly classified and $-\text{label} \cdot (\text{weight} \cdot \text{point} + \text{bias})$ for misclassified points.
8. Repeat the above till convergence is reached i.e. no train points are misclassified or till 300 iterations are completed.

Plot & Metrics



Training on Dataset A converged after 1 iteration, with 100% accuracy on the validation set.

Training on Dataset B converged after 13 iterations, with 99.167% accuracy on the validation set.

Since training converged on both datasets, the train points were linearly separable.

Since both datasets have the same mean for the two classes, the differences in epochs to convergence and test set accuracy is a direct result of the covariance of the 2 classes.

Dataset A had lower variance for each class, resulting in clusters for each class being far apart, and a decision boundary being found in a single step. (mathematically about 99.7% of points from both classes were within the radius of the mean till the midpoint connecting both means)

Dataset B had higher variance for each class resulting in some points of the clusters being very close together despite belonging to different classes, so the update drastically improved accuracy in the first iteration (due to most points being far apart), and then for the remaining iterations till convergence it was nudged towards the final boundary.